Book Tracker Project Documentation

1. Project Overview Document

Project Name: Shelf

Objective: A full-stack web application that allows users to discover books, manage personal reading lists, create custom collections, review books, and monitor reading activity.

Scope: The system includes user authentication, dashboards for users and administrators, profile management, search and filtering, book management operations, reviews, and collection management.

Target Users: General readers and administrators.

Entities and Roles:

- User: Registers, logs in, manages profile, tracks books and collections.
- Admin: Manages book catalog and administrative dashboards.
- Book: Represents books available in the system.
- Collection: Custom book groups created by users.
- Review: Ratings and comments on books.
- UserBook: Tracks reading status and favorites.
- Activity: Stores user activity history.

2. Functional Requirements Document (FRD)

User Authentication: Users register and log in securely using email and password. The system returns a JWT token for authenticated sessions.

Dashboard Access: Authenticated users can view personalized statistics and quick actions.

Profile Management: Users can view and edit their personal information and profile image.

Book Catalog Management: Users can browse books and administrators can add and delete books.

Book Details: Users can view detailed information, ratings, and reviews for each book.

Collection Management: Users can create and delete custom collections and add or remove books.

Review System: Users can submit ratings and add or remove comments for books.

Reading Tracker: Users can mark books as reading, completed, or want-to-read, and set favorites.

Activity Tracking: The system stores significant user actions.

Administrative Functions: Administrators access protected routes for catalog and dashboard management.

3. Technical Requirements Document (TRD)

Architecture: MERN-style application (React frontend, Node.js/Express backend, MongoDB database).

Frontend: React, Vite, React Router, Formik, Bootstrap.

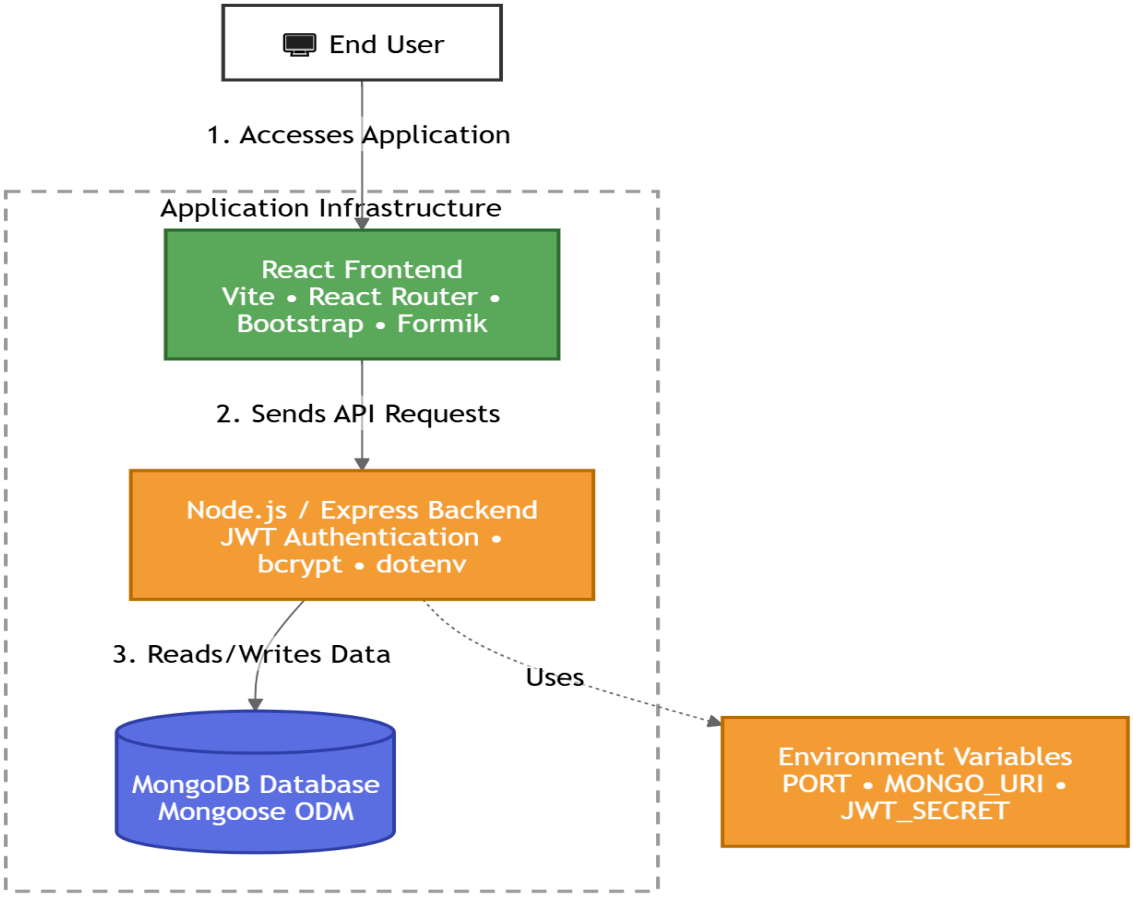
Backend: Node.js, Express, JWT, bcrypt, dotenv, Mongoose.

Database: MongoDB using Mongoose ODM.

Authentication: JWT tokens with protected middleware.

Infrastructure: Node.js runtime, MongoDB database server, environment variables (PORT, MONGO_URI, JWT_SECRET).

System Architecture Diagram:



4. Database Schema Documentation

User: Stores first_name, second_name, email, password, profile_image, gender, is_admin, and timestamps.

Book: Stores title, author, cover_image, published_year, language, description, categories, pages, rating, total_reviews, total_reads, and timestamps.

Review: References userId and bookId, and stores rate, comments, and timestamps.

Collection: References userId and stores name, description, an array of books, and timestamps.

UserBook: References userId and bookId, and stores status, isFavourite, and timestamps.

Activity: Stores type, user, book, and timestamps.

Relationships:

- One User can create many Collections.
- One User can have many UserBook records.
- One User can create many Reviews.
- One User can create many Activities
- One Collection contains many Books.
- One Book can appear in many Collections, UserBook records and Reviews.
- One Activity can reference one Book
- One User can track many Books through UserBook records.
- One Book can be tracked by many Users through UserBook records.

5. API Documentation

Description

This is an API for the Shelf Book Tracker application, allowing users to authenticate, manage personal book collections, log reading activity, and access a global library of books.

Base URL

The base URL for all API requests is:

`http://localhost:5000/api`

Auth Endpoints

```
POST /auth/register
```

Registers a new user and returns a token for secure access.

Parameters

- *FirstName (required) : the users first name.*
- *SecondName (required) : the user's second name.*
- Email(required) : the user's email.
- *password (required) : the users password.*
- gender (requird): the user's gender.

Response

Returns a JSON object containing the user's profile data:

- _id: The unique identifier of the user.
- First name: The user's first name.
- Second name: The user's second name.
- email: The user's email address.
- gender: The user's gender.
- is_admin: Boolean indicating if the user has administrative privileges.

Example

Request:

```
Post /api/user/me
```

Response:

JSON


```
{
"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoiNmE
wNzc5YTYzMTBjNzQ5M2JhYjRlYmFiIiwiaWF0IjoxNzc4ODc0NzkwLCJleHAiOjE3
ODE0NjY3OTB9.bAiMTVABT0anE7pc2Vpp9i6lDAKxxA9ls80KG32UGro",
"user":{
     "_id": "60f7a9b8e4b0c2a1d4e5f6g7",
     "first_name": "Sarah",
     "second_name": "Smith",
     "email": "sarah@example.com",
     "gender": "female",
     "is_admin": false
```

```
  }
}
```

`POST /auth/login`

User login to authenticate and receive a secure access token.

Parameters

- Email(required) : the user's email.
- *password (required) : the users password.*

Response

Returns a JSON object containing the authorization token and the user's profile data:

- token: The JWT used for authorization in subsequent requests.
- _id: The unique identifier of the user.
- First name: The user's first name.
- Second name: The user's second name.
- email: The user's email address.
- gender: The user's gender.
- is_admin: Boolean indicating if the user has administrative privileges.

Example

Request:

```
POST /api/auth/login

{ "email": "john@example.com",
  "password": "securePassword123"
}
```

Response:

```
JSON

{ "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoiNmEwNzc5Y",
"user": {
 "id": "60f7a9b8e4b0c2a1d4e5f6g7",
 "first_name": "Sarah",
 "second_name": "Smith",
```

```
 "email": "user@example.com",
 "gender": "female",
 "is_admin": false
 }
}
```

Collections Endpoints

`GET /collections`

Retrieves all custom and reading-status collections for the authenticated user.

Parameters

- *None (Requires Bearer Token in Authorization header).*

Response

Returns a JSON array of collection objects. Each object contains:

- _id: The unique identifier of the collection (or a status string like "reading", "completed", "want-to-read" for default tracking collections).
- userId: The unique identifier of the user (applicable to custom collections).
- name: The title of the collection.
- books: An array of detailed book objects contained within this collection.
- createdAt: The timestamp of when the collection was created.
- updatedAt: The timestamp of when the collection was last updated.

Example

Request:

```
GET /api/collections
```

Response:

```
JSON

[
  {
    "_id": "6a04e9364e50d030d0a74d92",
    "userId": "69fce6ae60a6cbe050736605",
    "name": "mido",
    "books": [
      {
        "cover_image": {
          "small": "https://covers.openlibrary.org/b/id/5-S.jpg",
          "medium":"https://covers.openlibrary.org/b/id/5-L.jpg",
```

```
          "large": "https://covers.openlibrary.org/b/id/5-L.jpg"
        },
        "_id": "69fe60f5f81893c1d4c136e3",
        "title": "The Hobbit",
        "author": "J.R.R. Tolkien",
        "published_year": 1878,
        "language": "English",
        "categories": [
          "Fantasy",
          "Adventure"
        ],
        "rating": 3.9
      }
    ],
    "createdAt": "2026-05-13T21:12:22.924Z",
    "updatedAt": "2026-05-13T22:24:18.608Z"
  },
  {
    "_id": "reading",
    "name": "Currently Reading",
    "books": []
  }
```

`POST /collections`

Creates a new custom book collection for the authenticated user.

Parameters

- *name (required) : The title of the new collection.*

Response

Returns a JSON object representing the newly created collection containing:

- _id: The unique identifier of the new collection.
- userId: The unique identifier of the user who created the collection.
- name: The title of the collection.
- books: An array initialized for adding books (starts empty).
- createdAt: The timestamp of when the collection was created.
- updatedAt: The timestamp of when the collection was last updated.
- __v: The internal database version key.

Example

Request:

```
POST /api/collections
{
  "name": "name"
}
```

Response:

```
JSON

{
  "userId": "69fce6ae60a6cbe050736605",
  "name": "name",
  "books": [],
  "_id": "6a078296d4b73b0f8d368cf4",
  "createdAt": "2026-05-15T20:31:18.739Z",
  "updatedAt": "2026-05-15T20:31:18.739Z",
  "__v": 0
}
```

`POST /collections/book/{bookId}`

Creates a new custom collection and immediately adds a specific book to it.

Parameters

- bookId (path, required) : The unique identifier of the book to add.
- name (body, required) : The name of the newly created collection.

Response

Returns a JSON object representing the newly created collection containing:

- _id: The unique identifier of the new collection.
- userId: The unique identifier of the user who created the collection.
- name: The title of the collection.
- books: An array initialized for adding books (starts empty).
- createdAt: The timestamp of when the collection was created.
- updatedAt: The timestamp of when the collection was last updated.
- __v: The internal database version key.

Example

Request:

```
POST /api/collections/book/69fe60f5f81893c1d4c136df
{
```

```
  "name": "collection"
}
```

Response:

```
JSON

{
  "userId": "69fce6ae60a6cbe050736605",
  "name": "colle",
  "books": [
    "69fe60f5f81893c1d4c136df"
  ],
  "_id": "6a078454a58c3d07f39b4f6a",
  "createdAt": "2026-05-15T20:38:44.577Z",
  "updatedAt": "2026-05-15T20:38:44.577Z",
  "__v": 0
}
```

`GET /collections/{id}`

Retrieves all custom collections for the authenticated user that contains a specific book.

Parameters

- bookId (path, required) : The unique identifier of the book you are checking.

Response

Returns a JSON array of collection objects. Each object contains:

- id: The unique identifier of the collection.
- name: The title of the collection.
- alreadyIn: A boolean (true or false) indicating if the specified book is already saved inside this collection.

Example

Request:

```
GET /api/collections/69fe60f5f81893c1d4c136df
```

Response:

```
JSON

[
  {
    "id": "6a04e9364e50d030d0a74d92",
    "name": "mido",
```

```
    "alreadyIn": true
  },
  {
    "id": "6a06362da238b42ad96169f5",
    "name": "reads",
    "alreadyIn": false
  },
  {
    "id": "6a063c277df46598bd43019a",
    "name": "the best",
    "alreadyIn": false
  }
]
```

`DELETE /collections/{id}`

Deletes a specific custom collection by its ID.

Parameters

- id (path, required) : The unique identifier of the collection you want to delete.

Response

Returns a JSON object with a success confirmation message:

- message: Success confirmation message.

Example

Request:

```
DELETE /api/collections/6a078454a58c3d07f39b4f6a
```

Response:

```
JSON

{
  "message": "Collection deleted successfully"
}
```

`POST /collections/addBook/{id}`

Adds a specific book to an existing custom collection.

Parameters

- id (path, required) : The unique identifier of the collection you are adding the book to.
- bookId (body, required) : The unique identifier of the book you want to add.

Response

Returns a JSON object with a success confirmation message:

- message: Success confirmation message.

Example

Request:

```
POST /api/collections/addBook/6a0756cb0430209dcc86abfa

{
  "bookId": "69fe60f5f81893c1d4c136e7"
}
```

Response:

```
JSON

{
  "message": "Book added to collection"
}
```

`DELETE /collections/removeBook/{id}`

Removes a specific book from a custom collection.

Parameters

- id (path, required) : The unique identifier of the collection.
- bookId (body, required) : The unique identifier of the book you want to remove.

Response

Returns a JSON object with a success confirmation message:

- message: Success confirmation message.

Example

Request:

```
DELETE /api/collections/removeBook/6a0756cb0430209dcc86abfa

{
  "bookId": "69fe60f5f81893c1d4c136e0"
}
```

Response:

```
JSON

{
  "message": "Book removed from collection"
}
```

User Endpoints

`GET /user/me`

Retrieves the profile information for the currently authenticated user.

Parameters

- None (Requires Bearer Token in Authorization header)

Response

Returns a JSON object containing the user's profile data:

- _id: The unique identifier of the user.
- first_name: The user's first name.
- second_name: The user's second name.
- email: The user's email address.
- profile_image: The URL or path to the user's profile picture.
- gender: The user's gender.
- is_admin: Boolean indicating if the user has administrative privileges.
- createdAt: The timestamp of when the account was created.
- updatedAt: The timestamp of when the profile was last updated.
- __v: The internal database version key.

Example

Request:

```
GET /api/user/me
```

Response:

```JSON
{
  "_id": "69fce6ae60a6cbe050736605",
  "first_name": "mohamed",
  "second_name": "ahmed",
  "email": "mohamed@gmail.com",
  "profile_image":
"http://localhost:5000/assets/profile_imgs/69fce6ae60a6cbe0507366
05-1778859740484.jpg",
  "gender": "male",
  "is_admin": false,
  "createdAt": "2026-05-07T19:23:27.006Z",
  "updatedAt": "2026-05-15T15:42:20.962Z",
  "__v": 0
}
```

```
PUT /user/me
```

Updates the profile information for the currently authenticated user. This endpoint uses multipart/form-data to support image uploads.

Parameters

- firstName (optional) : The user's updated first name.
- secondName (optional) : The user's updated second name.
- email (optional) : The user's updated email address.
- profile_image (optional) : A binary file representing the user's new profile picture.

Response

Returns a JSON object containing the updated user profile data:

- _id: The unique identifier of the user.
- first_name: The user's first name.
- second_name: The user's second name.
- email: The user's email address.
- profile_image: The URL or path to the user's profile picture.

- gender: The user's gender.
- is_admin: Boolean indicating if the user has administrative privileges.
- createdAt: The timestamp of when the account was created.
- updatedAt: The timestamp of when the profile was last updated.
- __v: The internal database version key.

Example

Request:

`PUT /api/user/me`

Response:

```
{
  "id": "69fce6ae60a6cbe050736605",
  "first_name": "Mohamed",
  "second_name": "ahmed",
  "email": "mohamed@gmail.com",
  "gender": "male",
  "is_admin": false,
  "profile_image":
"http://localhost:5000/assets/profile_imgs/69fce6ae60a6cbe0507366
05-1778859740484.jpg",
  "createdAt": "2026-05-07T19:23:27.006Z"
}
```

Books Endpoints

`POST /books`

Creates a new book in the global catalog. This endpoint is restricted to administrative users.

Parameters

- title (required) : The title of the book.
- author (required) : The author of the book.
- published_year (optional) : The year the book was published.
- categories (optional) : An array of strings representing the book's genres.
- pages (optional) : The total number of pages in the book.
- rating (optional) : The initial rating of the book.

Response

Returns Confirmation message:

- _message: Success confirmation message.

Example

Request:

```
POST /api/books

{
  "title": "Neuromancer",
  "author": "William Gibson",
  "published_year": 1984,
  "categories": [
    "Cyberpunk",
    "Science Fiction"
  ],
  "pages": 271,
  "rating": 4.5
}
```

Response:

```
JSON

{
  "message": "Language is required"
}
```

`GET /books`

Retrieves a complete list of all books available in the global catalog.

Parameters

- *None (Requires Bearer Token in Authorization header).*

Response

Returns a JSON array of book objects. Each object contains:

- _id: The unique identifier of the new book.
- title: The title of the book.
- author: The author of the book.
- cover_image: An object containing the book's cover image data.
- published_year: The year the book was published.

- categories: An array of genre tags.
- pages: The total number of pages.
- rating: The rating of the book.
- total_comments: The total number of comments or reviews (initialized to 0).

Example

Request:

```
GET /api/books
```

Response:

JSON


```
[
  {
    "_id": "69fe60f6f81893c1d4c1370a",
    "title": "Neuromancer",
    "author": "William Gibson",
    "cover_image": {},
    "published_year": 1984,
    "categories": [
      "Cyberpunk",
      "Science Fiction"
    ],
    "pages": 271,
    "rating": 4.5,
    "total_comments": 2
  },
  {
    "_id": "69fe60f5f81893c1d4c136e0",
    "title": "1984",
    "author": "George Orwell",
    "cover_image": {},
    "published_year": 1949,
    "categories": [
      "Dystopian",
      "Science Fiction"
    ],
    "pages": 328,
    "rating": 4.8,
    "total_comments": 15
  }
]
```

`GET /books/search`

Retrieves a paginated list of books from the global catalog, with optional search filtering.

Parameters

- *search (optional) : A keyword string to filter the books (e.g., by title or author).*
- *page (optional) : The specific page number of results to retrieve.*
- *limit (optional) : The maximum number of books to return per page.*

Response

Returns a JSON object containing the pagination metadata and the array of matching books:

- books: An array of detailed book objects matching the query.
- currentPage: The current page number being viewed.
- totalPages: The total number of available pages based on the limit.
- totalBooks: The total count of books matching the search criteria.

Example

Request:

`GET /api/books/search?page=1&limit=2`

Response:

JSON


```
{
  "books": [
    {
      "cover_image": {
        "small": "https://covers.openlibrary.org/b/id/8259480-
S.jpg",
        "medium": "https://covers.openlibrary.org/b/id/8259480-
M.jpg",
        "large": "https://covers.openlibrary.org/b/id/8259480-
L.jpg"
      },
      "_id": "69fe60f6f81893c1d4c13710",
      "title": "American Gods",
      "author": "Neil Gaiman",
      "published_year": 2003,
      "language": "English",
      "description": "A fascinating read titled American
Gods...",
      "categories": [
```

```
        "Fantasy",
        "Mythology"
      ],
      "pages": 563,
      "rating": 3.7,
      "total_reads": 135,
      "total_reviews": 0,
      "createdAt": "2026-05-08T22:17:26.029Z",
      "updatedAt": "2026-05-08T22:17:26.029Z",
      "__v": 0
    },
    {
      "cover_image": {
        "small": "https://covers.openlibrary.org/b/id/8259479-
S.jpg",
        "medium": "https://covers.openlibrary.org/b/id/8259479-
M.jpg",
        "large": "https://covers.openlibrary.org/b/id/8259479-
L.jpg"
      },
      "_id": "69fe60f6f81893c1d4c1370f",
      "title": "Good Omens",
      "author": "Neil Gaiman",
      "published_year": 2002,
      "language": "English",
      "categories": [
        "Comedy",
        "Fantasy"
      ],
      "pages": 871,
      "rating": 4.7,
      "total_reads": 197,
      "total_reviews": 0
    }
  ],
  "currentPage": 1,
  "totalPages": 5,
  "totalBooks": 48
}
```

`DELETE /books/{id}`

Deletes a specific book from the catalog by its ID. This action is typically restricted to administrative users.

Parameters

- *id (path, required) : The unique identifier of the book you want to delete.*

Response

Returns a JSON object with a success confirmation message:

- message: Success confirmation message.

Example

Request:

```
DELETE /api/books/69fe60f6f81893c1d4c1370a
```

Response:

```JSON
{
  "message": "Book deleted successfully"
}
```

```
GET /books/{id}
```

Retrieves detailed information for a specific book from the global catalog using its unique ID.

Parameters

- *id (path, required) : The unique identifier of the book you want to retrieve.*

Response

Returns a JSON object containing the book's detailed information:

- _id: The unique identifier of the book.
- title: The title of the book.
- author: The author of the book.
- cover_image: An object containing the book's cover image data or URLs.
- published_year: The year the book was published.
- categories: An array of genre tags (e.g., "Cyberpunk", "Science Fiction").
- pages: The total number of pages in the book.
- rating: The current average rating of the book.
- total_comments: The total number of comments or reviews the book has received.

Example

Request:

`GET /api/books/69fe60f6f81893c1d4c1370a`

Response:

JSON

```
{
  "_id": "69fe60f6f81893c1d4c1370a",
  "title": "Neuromancer",
  "author": "William Gibson",
  "cover_image": {},
  "published_year": 1984,
  "categories": [
    "Cyberpunk",
    "Science Fiction"
  ],
  "pages": 271,
  "rating": 4.5,
  "total_comments": 2
}
```

`GET /popular`

Retrieves a list of the most popular books in the global catalog, ordered by their total number of reads.

Parameters

- *limit (optional) : The maximum number of popular books to return.*

Response

Returns a JSON array of book objects. Each object contains:

- _id: The unique identifier of the book.
- title: The title of the book.
- author: The author of the book.
- cover_image: An object containing the book's cover image data or URLs.
- published_year: The year the book was published.
- categories: An array of genre tags (e.g., "Cyberpunk", "Science Fiction").
- pages: The total number of pages in the book.
- rating: The current average rating of the book.
- total_reads: The total number of times the book has been marked as read.

Example

Request:

```
GET /api/books/popular?limit=1
```

Response:

JSON


```
[
 {
    "_id": "69fe60f6f81893c1d4c1370a",
    "title": "Neuromancer",
    "author": "William Gibson",
    "cover_image": {
      "small": "https://covers.openlibrary.org/b/id/8259474-
S.jpg",
      "medium": "https://covers.openlibrary.org/b/id/8259474-
M.jpg",
      "large": "https://covers.openlibrary.org/b/id/8259474-
L.jpg"
    },
    "published_year": 1984,
    "categories": [
      "Cyberpunk",
      "Science Fiction"
    ],
    "pages": 271,
    "rating": 4.5,
    "total_reads": 406
  }
]
```

UserBooks Endpoints

```
GET /user-books/status/{book_id}
```

Retrieves the authenticated user's current reading status and favorite toggle for a specific book.

Parameters

- book_id (path, required) : The unique identifier of the book you want to check.

Response

Returns a JSON object containing the user's current tracking data for the book:

- status: A string representing the current reading status (e.g., "Want to Read", "Currently Reading", "Read").
- isFavourite: A boolean indicating whether the user has marked this book as a personal favorite.

Example

Request:

```
GET /api/user-books/status/69fe60f5f81893c1d4c136e0
```

Response:

```
JSON

{
  "status": "Currently Reading",
  "isFavourite": true
}
```

```
POST /user-books/status/
```

Updates or sets the reading status for a specific book in the authenticated user's library.

Parameters

- book_id (required) : The unique identifier of the book you want to update.
- status (required) : The new reading status to apply (e.g., "Want to Read", "Currently Reading", or "Read").

Response

Returns a JSON object with a success confirmation message:

- message: Success confirmation message indicating the status was updated.

Example

Request:

```
POST /api/user-books/status

JSON

{
  "book_id": "69fe60f5f81893c1d4c136e0",
  "status": "Read"
}
```

Response:

```JSON
{
  "message": "Status updated"
}
```

`GET /user-books/reviews/{bookId}`

Retrieves all public reviews and ratings for a specific book.

Parameters

- bookId (path, required) : The unique identifier of the book you want to retrieve reviews for.

Response

Returns a JSON array of review objects. Each object contains:

- _id: The unique identifier of the review.
- userId: The identifier (or populated user object) of the person who wrote the review.
- bookId: The unique identifier of the book being reviewed.
- rate: The numeric rating given to the book (e.g., 1 to 5).
- comments: The text content of the user's review.
- createdAt: The timestamp of when the review was originally posted.
- updatedAt: The timestamp of when the review was last updated.

Example

Request:

```
GET /api/user-books/reviews/69fe60f5f81893c1d4c136e0
```

Response:

```JSON
[
  {
    "_id": "6a078454a58c3d07f39b4f6b",
    "userId": "69fce6ae60a6cbe050736605",
    "bookId": "69fe60f5f81893c1d4c136e0",
```

```
    "rate": 5,
    "comments": "An absolute masterpiece. The world-building is
incredible!",
    "createdAt": "2026-05-15T20:38:44.577Z",
    "updatedAt": "2026-05-15T20:38:44.577Z"
  },
  {
    "_id": "6a089565b69d4e18g40c5g7c",
    "userId": "6a04e9364e50d030d0a74d92",
    "bookId": "69fe60f5f81893c1d4c136e0",
    "rate": 4,
    "comments": "Really enjoyed it, though the pacing was a bit
slow in the middle.",
    "createdAt": "2026-05-16T10:15:20.123Z",
    "updatedAt": "2026-05-16T10:15:20.123Z"
  }
]
```

`POST /user-books/reviews/{bookId}`

Adds a new user review and rating to a specific book in the catalog.

Parameters

- bookId (path, required) : The unique identifier of the book you are reviewing.
- rating (body, required) : The numeric rating given to the book (e.g., a number from 1 to 5).
- comment (body, optional) : The text content of your review.

Response

Returns a JSON object with a success confirmation message:

- message: Success confirmation message indicating the review was saved.

Example

Request:

```
POST /api/user-books/review/69fe60f5f81893c1d4c136e0

{
  "rating": 5,
  "comment": "An absolute masterpiece. The world-building is
incredible!"
}
```

Response:

```
JSON

{
  "message": "Review added successfully"
}
```

`GET /user-books/rate/{bookId}`

Retrieves the average rating for a specific book in the catalog.

Parameters

- bookId (path, required) : The unique identifier of the book you want to retrieve the rating for.

Response

Returns a JSON object containing the book's rating:

- rating: The numeric average rating of the book.

Example

Request:

```
GET /api/user-books/rate/69fe60f5f81893c1d4c136e0
```

Response:

```
JSON

{
  "rating": 4.5
}
```

Activities & Dashboard Endpoints

`GET /activities/recent`

Retrieves a feed of recent activities related to the authenticated user.

Parameters

- *None (Requires Bearer Token in Authorization header).*

Response

Returns a JSON array of activity log objects. Each object contains:

- _id: The unique identifier of the activity record.
- type: A string representing the type of activity performed (e.g., "read", "add").
- user: The name of the user who performed the activity.
- book: The title of the book associated with the activity.
- createdAt: The timestamp of when the activity occurred.
- updatedAt: The timestamp of when the activity record was last updated.
- __v: The internal database version key.

Example

Request:

```
GET /api/activities/recent
```

Response:

JSON


```
[
  {
    "_id": "6a079b1f7ab601e2ba9d5125",
    "type": "read",
    "user": "Ammar Mohamed",
    "book": "War and Peace",
    "createdAt": "2026-05-15T22:15:59.454Z",
    "updatedAt": "2026-05-15T22:15:59.454Z",
    "__v": 0
  },
  {
    "_id": "6a079778143144385ac1b4ce",
    "type": "add",
    "user": "Admin",
    "book": "The Lightning Thief",
    "createdAt": "2026-05-15T22:00:24.419Z",
    "updatedAt": "2026-05-15T22:00:24.419Z",
    "__v": 0
  }
]
```

```
GET /dashboard/stats
```

Retrieves overview statistics and monthly activity data for the application dashboard.

Parameters

- *None (Requires Bearer Token in Authorization header)*.

Response

Returns a JSON object containing global application statistics and a monthly activity breakdown:

- stats: An object containing aggregate totals:
  - totalBooks: The total number of books in the catalog.
  - totalUsers: The total number of registered users.
  - topRatedAvg: The average rating across the platform's top-rated books.
- monthlyActivity: An array of objects representing activity trends over the recent
  - months. Each object contains:
  - month: The abbreviated month name (e.g., "Nov", "Dec").
  - count: The number of activities or engagements recorded during that month.

Example

Request:

```
GET /api/dashboard/stats
```

Response:

JSON

```
{
  "stats": {
    "totalBooks": 48,
    "totalUsers": 14,
    "topRatedAvg": 4.3
  },
  "monthlyActivity": [
    {
      "month": "Nov",
      "count": 6
    },
    {
      "month": "Dec",
      "count": 19
    },
    {
      "month": "Jan",
      "count": 22
    },
    {
      "month": "Feb",
      "count": 15
```

```
    },
    {
      "month": "Mar",
      "count": 11
    },
    {
      "month": "Apr",
      "count": 21
    },
    {
      "month": "May",
      "count": 14
    }
  ]
}
```